

NF Validation Research Strategies

This is a space for my findings from the papers that have been selected for a suggestion of improvements in yaksha prashna w.r.t NF validation and network verification aspects

Problem Statement

Yaksha Prashna is designed to prove the functional correctness of the NF from the bytecode. By achieving the one of the high time achievements on time constraints marking it as one of the pioneering works in the ebp industry it has its own limitations.

To address them we are doing a proper literature survey and research analysis to improve yaksha in the NF validation and Network verification perspectives. As far as we know that yaksha prashna has trouble executing stateful queries, we have only six fields that we try to incorporate and we also have the critical path explosion and lack of behavioural queries which leads to us having to search more on this part so that we get more idea on how to do the NF validation

We have done the literature survey of this domain and found a total of 89 papers in which after filtering our cause and relevance and impact we have come down to 16 papers being effective in strategising and working on our agenda. The spreadsheet link is given below and as we progress we are going to unveil the strategies and plan on innovating and inculcating newer strategies onto the previous improvements.

https://docs.google.com/spreadsheets/d/1co271FI6_rgMSXye8-byWxMWKNp_goEyqrY8yG3DUQo/edit?gid=0#gid=0

Paper ID	Title	Related? (Animesh)
J3	Heimdall: LLM + Symbolic Execution for eBPF C-to-Rust Migration Verification	Check once, title seems interesting.
D2	A Formally Verified NAT (VigNAT)	
D3	Vigor: Verifying Software Network Functions	
E8	A Type System for Information Flow in P4 Programs	Ignore
G5	Modular Safety Verification for Stateful Networks (Complexity Results)	Ignore
H3	Differential Testing of Click Middleboxes (Gravel/USENIX)	Ignore
F11	BeePL: Correct-by-Construction Kernel Extensions	Check this
J1	AppNet: Semantic-Aware Optimization of Application Network Functions	Check this
J2	eNetSTL: Safe and Efficient In-Kernel eBPF Network Function Library	Check what are the functionalities provided by the library.
J4	Demystifying Performance of eBPF Network Applications	Check this
I7	Intent-Based Management: An LLM-Centric Approach	Ignore
J5	FlowMage: LLM-Based Static Analysis for Stateful NF Chain Optimization	Check this for NF chain part
F9	Validating the eBPF Verifier via State Embedding (OSDI 2024)	Ignore
I3	Machine Learning-Based Anomaly Detection in NFV: Comprehensive Survey	Ignore
E3	Verifiable P4: Verified Modular Reasoning for Stateful P4 Programs	Ignore
D6	Automated Verification of Network Function Binaries (Klint)	Ignore
G4	NetSMC: A Custom Symbolic Model Checker for Stateful Network Verification	Check this
I6	Full-Lifecycle Intent-Driven Network Verification	Check this
G6	SLA-Verifier: Stateful and Quantitative Verification for Service	Ignore

Heimdall: LLM + Symbolic Execution for eBPF C-to-Rust Migration Verification

This paper address the source-level bugs that corrupt data and leak some prev traced events to userspace. Using this info they can recover data info which is normally hidden or made secure .by KASLR, (a cybersecurity mitigation that randomizes the memory locations of the kernel code). There were unreported leaks in 10 open source ebpf programs whose ring buffer or stack records had prev traced data. So heimdall is an automated pipeline tp harden such verifier accepted buggy programs. Now the question is does it come under NF validation? In a way,yes. Because at the end point it is still validating an NF program to not leak information and not have buggy programs but still it is majorly a compiler verification tool.

This is the purpose of the paper lets see what we can find out about it and the process that follows. Since they use LLMS to convert programs to convert it into rust Aya platform and then use symbolic execution and Z3 for equivalence checking, this can corroborate my neurosymbolic approach which is similar but i havent used claude because it costs me money. Can learn more on symbolic execution process to learn to improve.

this can be used as a **semantic enrichment layer**, not as the validator

This paper works on stripped,decompiled binaries.

We can find out what this NF does but it does not do the functional validation or correctness.

Acc to the diagram it converts source code to rust code and adds strategies and safety conditions and then compiles it into the bytecode and then it compares with the initial bytecode for similar check and then it passes.. Basically verification but what i feel is the semantic check ing is still on bytecode they are using symbolic excecution with angr(python based binary analysis platform)-use thaty to tweak in changes and have a backend extension to modify it and use it as symbolic execution process is interesting and it works

Equivalence checking is mathematical proof of similarity and z3 is there to show sat/unsat. Core thing to notice is how they are doing symbolic execution because if they are able to map it we can use this as reference tro also tweak in and build for nf validation. Interesting paper. Diagrams are below papere is interesting.

Heimdall: LLM + Symbolic Execution for eBPF C-to-Rust Migration Verification

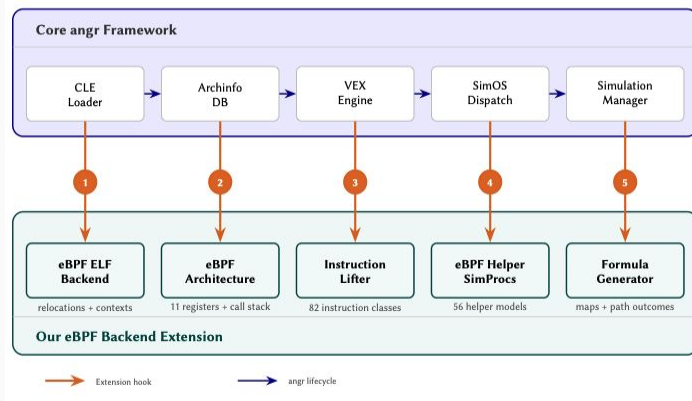
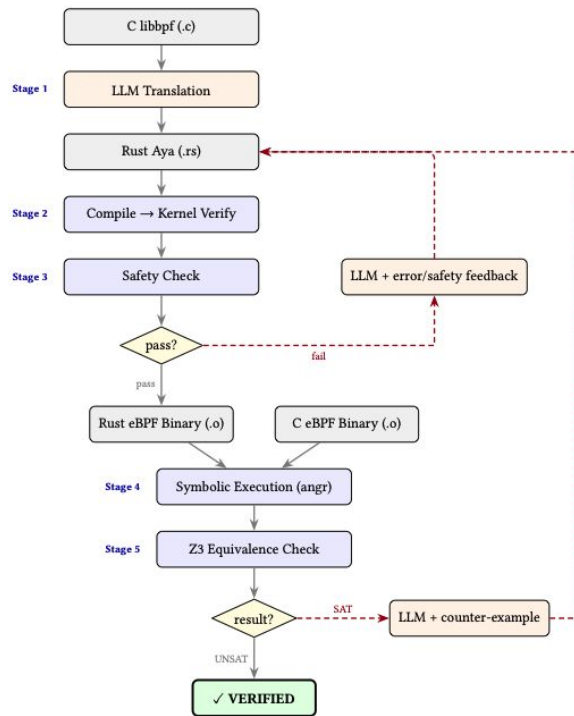
LLM Notes

[What it does]: Legacy eBPF C programs need to be migrated to safer Rust-eBPF, but verifying that migration does not change NF behavior is hard. Heimdall uses LLMs to guide the migration and Z3-based symbolic equivalence checking to prove behavioral preservation. [How it validates]: 1. Use LLM to translate eBPF C to Rust-eBPF. 2. Symbolically model both programs. 3. Use Z3 to prove functional equivalence. [eBPF Relevance]: Direct — validates eBPF NF behavioral correctness across implementation migration.

Approach wrt NF validation : LLM-guided C-to-Rust eBPF translation + Z3 behavioral equivalence: Use GPT-class LLMs to automatically translate legacy eBPF C code to Rust-eBPF. Symbolically model the execution behavior of both C and Rust versions. Apply Z3 SMT solver to prove that both versions produce identical packet decisions (accept/drop/redirect) and map state changes for all possible inputs.

eBPF NF behavioral equivalence across language migrations; packet decision preservation; map state change equivalence; migration correctness verification

HEIMDALL: Formally Verified Automated Migration of Legacy eBPF Programs to Rust



VigNat and Vigor

Verifying the Software Network Functions

Vigor and Vignat are typical papers that tell us how they can do the NF validation and network verification of NF s starting with NAT s and extending them to other NFs starting with firewall and loadbalancer. Why this could help us is because it aligns with our goals in understanding how an NF works, how a piece of coe works. May not be on a bytecode level but we can incorporate the logic into bytecode if this proves to be of more use and compatible to all NFs and most importantly solves our problem of NF chain. Lets deep dive into the paper.

VigNAT: Formally verified NAT

Vignat says thaty they have written a C program in such a way that it is crash free- memory safe and semantically correct as per RFC3022. It starts with addressing the problem of path explosion w.r.t symbolic excecution. Drawbacks is for multiple nfs or NF chain creates path explosion so they are they are splitting the NF s code into 2 parts :

1. Data structure library that keeps the difficult state which has to be formally proven for correctness.
2. Quick proving stateless code.

To combine these results they use a term called "lazy proofs"